

Evidencia de producto: GA7-220501096-AA5-EV02 API

Mario Hernando Gallo Gallego

GAES # 6

Servicio Nacional de Aprendizaje – (SENA)

Análisis y Desarrollo de Software – (3118493)

Andrés Rubiano Cucarían

Cali, Colombia

15 de marzo de 2026

1. Introducción

En el desarrollo de software moderno, las API (Application Programming Interface) permiten la comunicación entre diferentes sistemas o aplicaciones mediante servicios web. Para garantizar el correcto funcionamiento de estos servicios es necesario realizar pruebas que permitan validar las operaciones implementadas.

En esta evidencia se realiza el testing de la API desarrollada para el sistema de inventarios Proterquim, la cual fue construida utilizando el framework Spring Boot. El objetivo de estas pruebas es verificar el correcto funcionamiento de los endpoints implementados para la gestión de usuarios, específicamente los procesos de registro y autenticación.

Para realizar las pruebas se utilizó la herramienta Postman, que permite enviar solicitudes HTTP al servidor y analizar las respuestas obtenidas. De esta manera se valida que la API responde correctamente a las solicitudes realizadas por los clientes.

2. Objetivo

Realizar el testing de la API del sistema de inventarios Proterquim mediante la herramienta Postman, con el fin de verificar el funcionamiento de los endpoints desarrollados para el registro y autenticación de usuarios.

3. Alcance

Esta evidencia se enfoca en la validación de los servicios web implementados en la API del sistema de inventarios Proterquim. Las pruebas realizadas permiten verificar que los endpoints desarrollados responden correctamente a las solicitudes enviadas desde Postman.

Las pruebas incluyen:

- ✓ Ejecución del servidor backend.
- ✓ Verificación del endpoint de registro de usuarios.
- ✓ Validación del endpoint de autenticación (login).
- ✓ Comprobación de las respuestas generadas por la API.

Las pruebas se realizan en un entorno local utilizando una base de datos en memoria para fines académicos.

4. Tecnologías utilizadas

Para el desarrollo y testing de la API se utilizaron las siguientes tecnologías y herramientas:

- ✓ Java
- ✓ Spring Boot
- ✓ Spring Data JPA
- ✓ Base de datos H2 (en memoria)
- ✓ Postman
- ✓ Apache Maven
- ✓ Git y GitHub
- ✓ NetBeans IDE

5. Ejecución del proyecto

Para realizar las pruebas de la API fue necesario ejecutar el servidor backend del sistema de inventarios. La aplicación se ejecutó desde la terminal utilizando el siguiente comando:

```
./mvnw spring-boot:run
```

Este comando permite compilar el proyecto y ejecutar la aplicación desarrollada con Spring Boot, iniciando el servidor embebido en el puerto 8081.

Una vez iniciado el servidor, la API queda disponible para recibir solicitudes HTTP desde herramientas de prueba como Postman.

```
MINGW64/C:/Users/Mario Gallo/OneDrive/Escritorio/inventarios
[ jae ]
INFO [INFO] >>> spring-boot:3.2.5> (default-cli) > test-compile @ inventarios >>>
INFO [INFO] --- resources:3.3.1:resources (default-resources) @ inventarios ---
INFO [INFO] Copying 3 resource from src/main/resources to target/classes
INFO [INFO] Copying 3 resource from src/main/resources to target/classes
INFO [INFO] --- compiler:3.11.0:compile (default-compile) @ inventarios ---
INFO [INFO] Nothing to compile - all classes are up to date
INFO [INFO] --- resources:3.3.1:testResources (default-testResources) @ inventarios ---
INFO [INFO] skip non existing resourceDirectory C:/Users/Mario Gallo/OneDrive/Escritorio/inventarios/src/test/resources
INFO [INFO] --- compiler:3.11.0:testCompile (default-testCompile) @ inventarios ---
INFO [INFO] Nothing to compile - all classes are up to date
INFO [INFO] >>> spring-boot:3.2.5> (default-cli) < test-compile @ inventarios >>>
INFO [INFO] --- spring-boot:3.2.5> (default-cli) @ inventarios ---
INFO [INFO] Attaching agent: []

Spring
3.2.5

2026-01-10T17:08:14.887-05:00 INFO 11960 --- [protorquein-inventarios] main] c.p.i.ProtorqueinInventariosApplication Starting ProtorqueinInventariosApplication using Java 21.0.10 with PID 11960 (C:/Users/Mario Gallo/OneDrive/Escritorio/inventarios/target/classes started by Mario Gallo in C:/Users/Mario Gallo/OneDrive/Escritorio/inventarios)
2026-01-10T17:08:14.887-05:00 INFO 11960 --- [protorquein-inventarios] main] c.p.i.ProtorqueinInventariosApplication No active profile set, falling back to 1 default profile: 'default'
2026-01-10T17:08:14.887-05:00 INFO 11960 --- [protorquein-inventarios] main] c.p.i.ProtorqueinInventariosApplication Bootstrapping Spring Data JPA repositories in DEFALT mode
2026-01-10T17:08:14.887-05:00 INFO 11960 --- [protorquein-inventarios] main] c.p.i.ProtorqueinInventariosApplication Finished Spring Data repository scanning in 99 ms. Found 2 jpa repository interfaces.
2026-01-10T17:08:14.887-05:00 INFO 11960 --- [protorquein-inventarios] main] c.p.i.ProtorqueinInventariosApplication Tomcat initialized with port 8081 (http)
2026-01-10T17:08:14.887-05:00 INFO 11960 --- [protorquein-inventarios] main] c.p.i.ProtorqueinInventariosApplication Starting service [tomcat]
2026-01-10T17:08:14.887-05:00 INFO 11960 --- [protorquein-inventarios] main] c.p.i.ProtorqueinInventariosApplication Starting Servlet engine: [Apache Tomcat/10.1.20]
2026-01-10T17:08:14.887-05:00 INFO 11960 --- [protorquein-inventarios] main] c.p.i.ProtorqueinInventariosApplication Initializing Spring embedded webApplicationContext: initialization completed in 2909 ms
2026-01-10T17:08:14.887-05:00 INFO 11960 --- [protorquein-inventarios] main] c.p.i.ProtorqueinInventariosApplication Starting
2026-01-10T17:08:14.887-05:00 INFO 11960 --- [protorquein-inventarios] main] c.p.i.ProtorqueinInventariosApplication Added connection comdb: url=jdbc:h2:mem:inventariosdb user=SA
2026-01-10T17:08:14.887-05:00 INFO 11960 --- [protorquein-inventarios] main] c.p.i.ProtorqueinInventariosApplication Start completed
2026-01-10T17:08:14.887-05:00 INFO 11960 --- [protorquein-inventarios] main] c.p.i.ProtorqueinInventariosApplication No console available at /?2-console. Database available at 'jdbc:h2:mem:inventariosdb'
2026-01-10T17:08:14.887-05:00 INFO 11960 --- [protorquein-inventarios] main] c.p.i.ProtorqueinInventariosApplication Processing ServletContextInfo [name: default]
2026-01-10T17:08:14.887-05:00 INFO 11960 --- [protorquein-inventarios] main] c.p.i.ProtorqueinInventariosApplication Hibernate ORM core version 6.4.4.Final
2026-01-10T17:08:14.887-05:00 INFO 11960 --- [protorquein-inventarios] main] c.p.i.ProtorqueinInventariosApplication Second-level cache disabled
2026-01-10T17:08:14.887-05:00 INFO 11960 --- [protorquein-inventarios] main] c.p.i.ProtorqueinInventariosApplication No LoadTimeWeaver setup: ignoring JPA class transformer
2026-01-10T17:08:14.887-05:00 INFO 11960 --- [protorquein-inventarios] main] c.p.i.ProtorqueinInventariosApplication No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA)
2026-01-10T17:08:14.887-05:00 INFO 11960 --- [protorquein-inventarios] main] c.p.i.ProtorqueinInventariosApplication Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-01-10T17:08:14.887-05:00 INFO 11960 --- [protorquein-inventarios] main] c.p.i.ProtorqueinInventariosApplication spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering. Enable this setting by setting 'spring.jpa.open-in-view' to false in configuration
2026-01-10T17:08:14.887-05:00 INFO 11960 --- [protorquein-inventarios] main] c.p.i.ProtorqueinInventariosApplication Tomcat started on port 8081 (http) with context path ''
2026-01-10T17:08:14.887-05:00 INFO 11960 --- [protorquein-inventarios] main] c.p.i.ProtorqueinInventariosApplication Started ProtorqueinInventariosApplication in 8.218 seconds (process running for 8.945)
```

6. Desarrollo de la evidencia – Pruebas realizadas

A continuación se presentan las pruebas realizadas a los endpoints de la API utilizando Postman.

6.1 Prueba 1 – Registro de usuario

Para validar el funcionamiento del registro de usuarios se utilizó el endpoint correspondiente al servicio de registro.

Método HTTP

POST

Endpoint

http://localhost:8081/api/registro

Body enviado en formato JSON

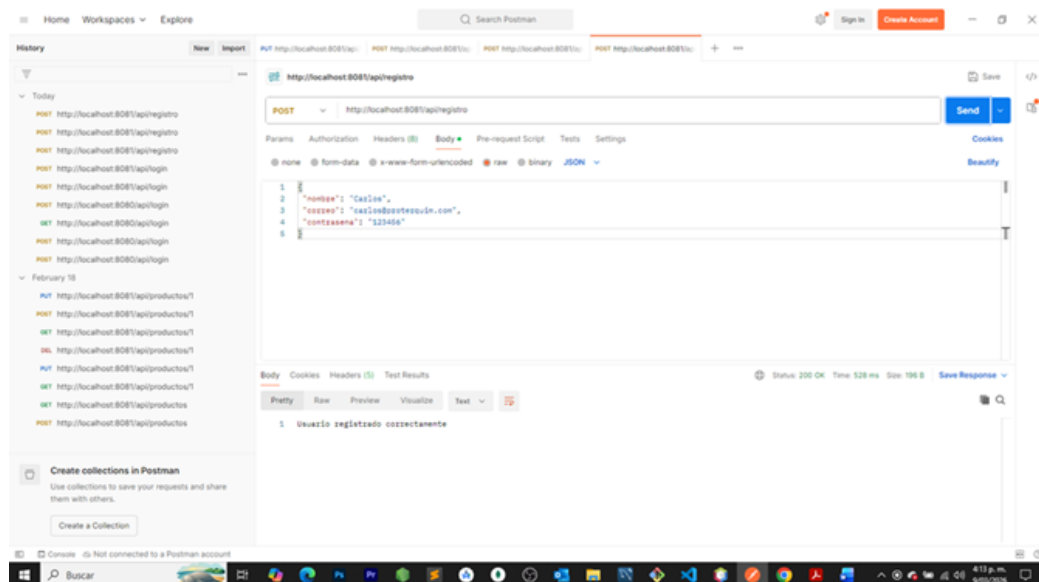
```
{  
  
  "nombre": "Carlos",  
  
  "correo": "carlos@proterquim.com",  
  
  "contrasena": "123456"  
}
```

Esta solicitud permite registrar un nuevo usuario en el sistema. Al enviar los datos del usuario, la API procesa la información y la almacena en la base de datos.

Como resultado, el sistema devuelve el mensaje:

Usuario registrado correctamente

lo que indica que el proceso de registro se realizó de forma exitosa.



6.2 Prueba 2 – Login de usuario

En esta prueba se validó el endpoint de autenticación del sistema, el cual permite verificar las credenciales de un usuario previamente registrado.

Método HTTP

POST

Endpoint

http://localhost:8081/api/login

Body enviado en formato JSON

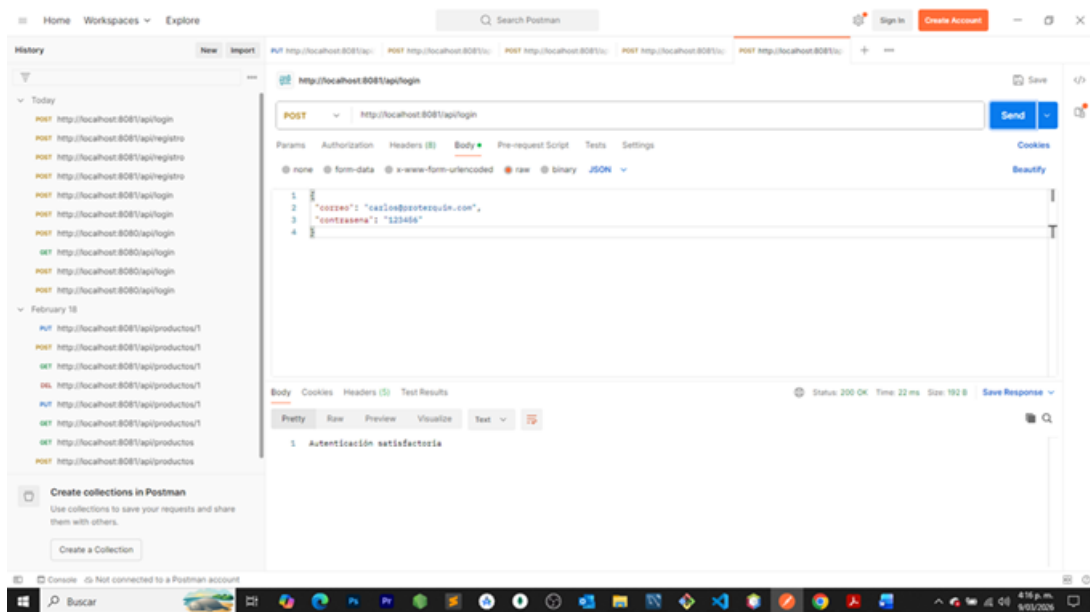
```
{  
  
  "correo": "carlos@proterquim.com" ,  
  
  "contrasena": "123456"  
}
```

Este endpoint recibe las credenciales del usuario y verifica si coinciden con la información almacenada en la base de datos.

Como respuesta, el sistema devuelve el mensaje:

Autenticación satisfactoria

lo que confirma que el proceso de autenticación funciona correctamente.



7. Documentación de ENDPOINTS de la API

A continuación se presentan los endpoints utilizados en la API para la gestión de usuarios.

Método	Endpoint	Descripción
Post	/api/registro	Permite registrar un nuevo usuario en el sistema
Post	/api/login	Permite autenticar un usuario registrado

Base URL del servidor

<http://localhost:8081>

8. Repositorio del proyecto

El código fuente del proyecto se encuentra almacenado en el siguiente repositorio de GitHub:

<https://github.com/MG2079/proterquim-inventarios>

En este repositorio se encuentra el backend desarrollado con Spring Boot, así como los archivos necesarios para la ejecución del sistema.

9. Conclusiones

El testing realizado permitió verificar el correcto funcionamiento de los endpoints desarrollados en la API del sistema de inventarios Proterquim.

Mediante el uso de la herramienta Postman fue posible enviar solicitudes HTTP al servidor y validar las respuestas generadas por la aplicación, comprobando que los servicios de registro y autenticación de usuarios funcionan correctamente.

Estas pruebas permiten garantizar que la API puede ser utilizada por otras aplicaciones o interfaces de usuario, facilitando la integración entre sistemas y asegurando la correcta gestión de usuarios dentro del sistema.

10. Bibliografías

- ✓ Spring Boot. (2024).
- ✓ <https://spring.io/projects/spring-boot>
- ✓ Postman. (2024).
- ✓ <https://www.postman.com/>
- ✓ Apache Maven Project. (2024).
- ✓ <https://maven.apache.org/>
- ✓ Documentación oficial de Spring Framework.
- ✓ <https://docs.spring.io>